

節 1 / 共 4 節

從對話到派工

AI 幫你寫程式，跟你問 AI 問題，不是同一件事

這兩天，我們要做一件事

把 LLM 從一個「聊天玩具」，變成一個 **你敢把工作交給它、而且知道怎麼檢查它** 的系統。

重點 這門課不教 transformer、不教訓練、不教數學。我們只做一件事：**把它接進真實的工作流程裡，然後確認它沒騙你。**

LLM 只是個會猜下一個字的東西。
讓它變成能交付的系統的，
是你在它外面搭的回饋迴路。

四節課，就是把迴路搭起來

節次	問題	你會學到
節 1	派工跟問答差在哪？	agent 能做什麼、你要給它什麼
節 2	它有什麼工具？	harness：初始輸入檔、skill、MCP、hook
節 3	它看得到什麼？	RAG、把 LLM 放進真實產品
節 4	你憑什麼相信它？	輸出驗證、成本、資安

實務 前三節在**放大**它的能力，第四節在**收斂**它的不確定性。只做前面 → 做出一個沒人敢用的東西；只做後面 → 什麼都沒做出來。

這門課怎麼上

講述 4 成、動手 6 成。你會一直在敲鍵盤。

規則

- 每節 3 小時，中間休息 10 分鐘
- 共 9 個 Lab，每個都有**基礎**與**進階**兩層
- 卡住超過 5 分鐘就舉手，不要自己硬撐
- 做完基礎的請去幫旁邊的人（這是最有效的學習）

你需要準備

- 電腦已預裝好環境（課前完成）
- 一個 OpenCode Zen 帳號（等一下註冊）
- 一個能記東西的地方（記事本、筆記軟體都行）

注意 免費模型額度有限，不要拿它閒聊，額度要留給 Lab。

\$ whoami

陳冠惟 Guan-Wei Chen

現在

- 陽明交通大學 網路工程研究所 碩二
- Panglotlabs 後端軟體工程師

經歷

- 東海資工 41th
- 東海大學網管小組成員
- 工業技術研究院 實習

證照 & 榮譽

- CEH (Certified Ethical Hacker)
- AIS3 EOF 決賽



1-A

它的邊界，與 agent 怎麼補

先看模型本身做得到什麼、做不到什麼。

「怎麼問」這一節不教

網頁版的使用經驗，已經涵蓋了這一層。

- 問得模糊，答得就模糊
- 給例子比給描述準
- 講清楚格式，它就照做

所以這一節不花時間教你怎麼問。
我們直接跳到你還沒用過的那個東西。

情境一：用 AI 寫 code

問「Rust 怎麼產生亂數」。它照**訓練時看過的資料**回答——而那份資料有時效。

```
``toml
[dependencies]
rand = "0.8"
``

fn main() {
    let mut rng = thread_rng(); // 获取随机数生成器

    // 1. 生成整数 (均匀分布)
    let n: i32 = rng.random();           // 任意 i32
    let n: usize = rng.random();         // usize
    let n: u8 = rng.random();             // u8
    let n = rng.gen_range(1..=10);        // 1 到 10 (包含两端)
```

rand = "0.8" 最新是 0.9.5 | thread_rng() \ gen_range() 0.8 的名字 | **rng.random() 是 0.9 的名字**

真的拿去編譯，兩邊都不對

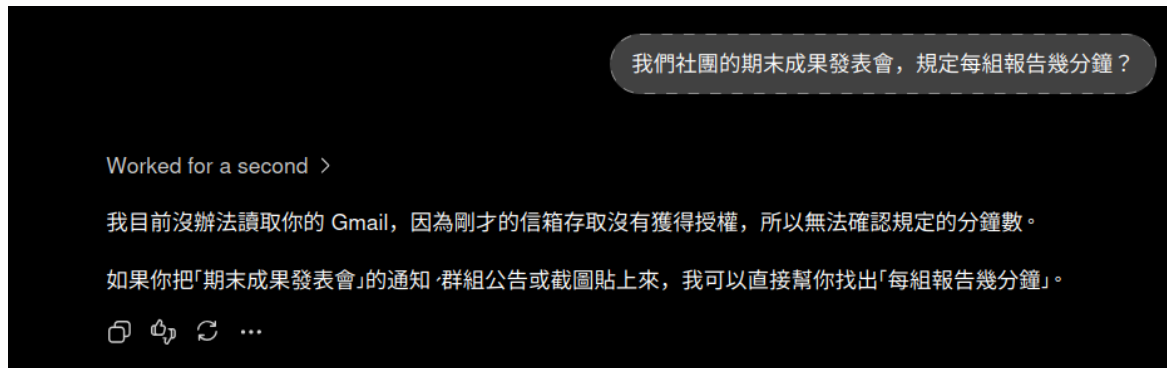
情境一（續）：同一段程式碼，配不同版本。

依賴宣告	cargo build 的結果
rand = "0.8"	✗ 編譯失敗 —— E0599: ThreadRng 沒有 random 這個 method
rand = "0.9"	能編，但 3 個 deprecated 警告

注意 它混用了兩個版本的 API，讀 code 讀不出來 —— cargo build 一秒就講了。

情境二：叫 AI 處理你自己的東西

問一個社團內部才知道的規定。**但它沒有這份資料。**



重點 注意最後那句：「你把公告貼上來，我就能幫你找出。」

注意 貼一次可以。但**你得先知道答案在哪一份文件**，找出來、貼進去 —— 每問一次做一遍，**關掉分頁還要重來**。

這門課教的每樣東西，都對應一種你會遇到的狀況

熱門、穩定、通用的東西它很準。會卡住的就這幾種情況。

你想做的事	會遇到什麼	用什麼解
用一個沒用過的套件	版本落後、API 搬家	讓它去查、去跑編譯 節 2
要它照專案的風格寫	它不知道你的規矩	AGENTS.md 節 2
同一套流程一直重跑	每次都要重講一遍	Skill 節 2
要它碰內部系統／私有文件	它拿不到	工具 / MCP、RAG 節 2、3
要它產出程式要吃的資料	算錯、欄位對不上	輸出驗證 節 4

注意 另一個獨立的問題：**同樣輸入不保證同樣輸出**，「試過一次」不算數。

補救的東西，全都在模型外面

同樣的模型，agent 多了三件事。

	網頁版	Agent
看得到什麼	只有貼進去的字	整個專案 ，它自己去讀檔
能做什麼	只能回字	真的動手 ：跑編譯、跑測試、看錯誤
記得什麼	跨 session 要重講一遍	對話一樣不跨 session，但 設定檔每次自動載入

重點 網頁版是問答，**agent 是派工**。你不再是複製貼上的中間人。

實務 **模型沒變，變的是它外面**。接下來三節，就是把這三件事一個一個接上去。

Lab 0

環境健檢

Lab 0 1. 裝 pi

25 分鐘

目標 到 <https://pi.dev/> 複製對應的指令裝 pi。

Windows 11

```
powershell -c "irm https://pi.dev/install.ps1 | iex"
```

它會順便問你要不要裝 Node —— 說要。

Linux / macOS

```
curl -fsSL https://pi.dev/install.sh | sh
```

```
sixsquare@sixsquare-laptop:~$ curl -fsSL https://pi.dev/install.sh | sh

Pi
Pi Installer
There are many agent harnesses but this one is yours

Install command:
npm install -g --ignore-scripts --min-release-age=0 @earendil-works/pi-coding-agent

Choose an action:
y Install Pi (default)
n Do nothing

Will install Pi.

✓ install complete

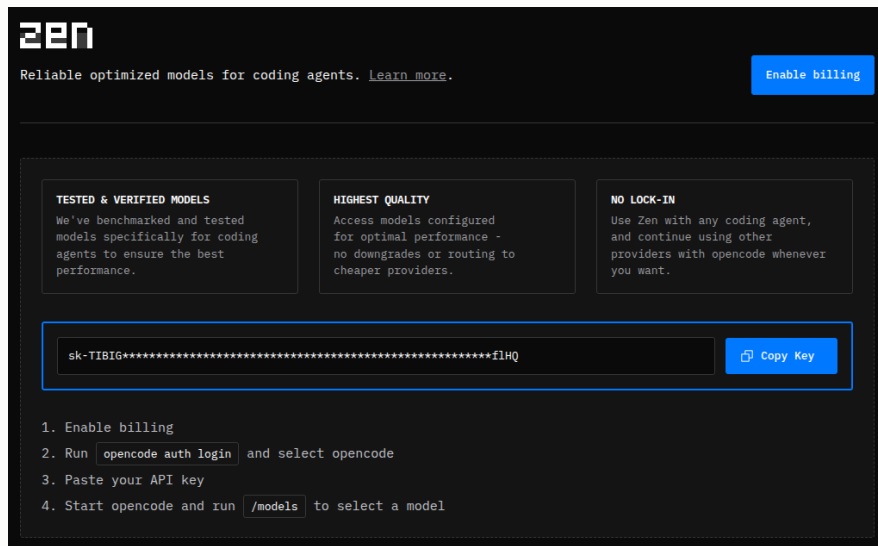
Pi was installed successfully.

Run it with: pi
sixsquare@sixsquare-laptop:~$
```

實務 驗收 開一個新的終端機，打 pi 叫得出來。裝完不換視窗，PATH 不會生效。

Lab 0 2. 拿一把 key

目標 到 OpenCode Zen 註冊，複製 API key。



開 <https://opencode.ai/auth> 註冊／登入，複製 API key。

重點 不需要填信用卡。我們全程只用免費模型。

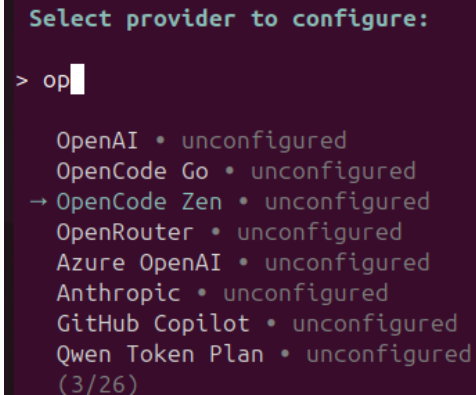
注意 免費模型會記錄你送出的內容。lab 素材都是虛構的——不要餵真實個資。

Lab 0 3. 把 key 接上 pi

目標 在 pi 裡面用 `/login`，不用設環境變數。

1. 終端機打 `pi`，進去之後輸入 `/login`
2. 選 **Sign in with an API key**
3. 選 **OpenCode Zen**
4. 貼上剛剛複製的 key

實務 驗收 pi 開起來，右下角看得到模型名稱，不再出現 `No models available`。

A terminal window with a dark purple background. At the top, it says "Select provider to configure:". Below that, the prompt "> op" is followed by a cursor. A list of providers is shown, each with a status "unconfigured". The providers are: OpenAI, OpenCode Go, OpenCode Zen (which is highlighted with a blue arrow), OpenRouter, Azure OpenAI, Anthropic, GitHub Copilot, and Qwen Token Plan. At the bottom of the list, it says "(3/26)".

```
Select provider to configure:
> op
OpenAI • unconfigured
OpenCode Go • unconfigured
→ OpenCode Zen • unconfigured
OpenRouter • unconfigured
Azure OpenAI • unconfigured
Anthropic • unconfigured
GitHub Copilot • unconfigured
Qwen Token Plan • unconfigured
(3/26)
```

實務 登入一次就好。 之後所有 lab 都走 pi，不用設環境變數。

Lab 0 4. 確認它會回話

目標 跑通一次，確認可以用。

```
pi --provider opencode --model mimo-v2.5-free \  
-p "用繁體中文回我一句 hello"
```

注意 每次都要指定 `--model`，不然它會用預設模型——那可能是付費的。

實務 驗收 終端機看得到模型的回覆。

備用模型：hy3-free、nemotron-3.5-lightning-free

1-C

Agent 時代的四個名詞

prompt / context / harness / loop engineering

同一個任務，四個層次

任務：「幫我把這 **50 份** CSV 整理成月報表」

名詞	白話	在這個任務裡長什麼樣
Prompt engineering	怎麼問	「整理成報表」→「輸出 markdown 表格，欄位 A/B/C，數字兩位小數」
Context engineering	給它看什麼	欄位說明、去年的報表當範例、哪幾欄可以忽略
Harness engineering	給它什麼工具與規矩	讀檔工具、只能寫進 output/、寫完自動跑檢查
Loop engineering	誰決定下一輪	一份做完自己挑下一份， 50 份都好了才停

前兩層你已經在做了

Prompt engineering	把模糊的期待換成明確的規格。給例子比給描述有效， 給反例又更有效
Context engineering	問題常常不是「怎麼問」，是 你根本沒給 。該給相關檔案、規格、範例、先前的決定

注意 這兩層的共通問題：**每次都要重講一遍**。你在網頁版一定有過「又要再解釋一次這個專案」的經驗。

實務 節 2 的第一件事，就是讓這兩層**持久化**——講一次，之後每次都生效。

後兩層是今天開始的新東西

Harness engineering	給它什麼 工具與規矩 ：能讀檔、只能改哪裡、做完要通過什麼檢查
Loop engineering	取代「一直按 enter 的那個人」 ：由系統決定何時再叫模型一次、什麼算完成、什麼時候停

重點 Harness 問「它需要什麼環境」（**一個回合**的事）；
Loop 問「什麼循環讓它朝目標前進、何時該停」（**很多回合**的事）。

實務 Loop 的關鍵在於**停止條件必須是客觀的**：編譯過了、測試綠了、清單空了。

多數人花 90% 的時間在調 prompt 。
實務上 80% 的效果，
來自後面那三層。

而你已經把 90% 的時間花在第一層了。接下來三節，我們補上另外三層。

Lab 1

第一次派工

不要再當複製貼上的中間人。

Lab 1 用 agent 做一個真的功能

60 分鐘

目標 體感「派工」跟「問答」的差別，並且看清楚它到底做了什麼。

素材： labs/lab1-first-task/habit/ —— 一個能跑的小 CLI，風格很明確。

1. `bash init.sh` 把它變成 git repo，等一下才看得到改動
2. 打 pi 開起來，用 `/model` 選 `mimo-v2.5-free`
3. 丟一個需求給它，**用你平常跟網頁版講話的方式**就好
4. **看著它做** —— 它讀了哪些檔？跑了什麼指令？改了幾次？
5. 它說做完之後，`git diff` **逐行看**
6. 填「觀察記錄表」（下一頁）

重點 驗收 專案多了一個能跑的功能，而且你填得出記錄表。

看著它做，不等於知道它做了什麼

所以第 5 步的 `git diff` 不能跳。

畫面上跑過去的是**它想讓你看到的摘要**。

git diff 是**它實際做了什麼**。

- 它會說「已新增 streak 指令」—— 但沒說它順手改了 ui.ts
- 它會說「已處理錯誤情況」—— 但沒說用的是 throw 而不是專案的 fail()

注意 這是全課第一次出現「不要相信敘述，去看產出」。節 4 整節都在講這件事。

Lab 1 觀察記錄表

這張表是今天最重要的產出。節 2 一開始就要用它。

問題	第 1 次（隨便講）	第 2 次（講清楚）
它改了幾個檔？		
它做了什麼 你沒要求 的事？		
它 漏 了什麼你以為理所當然的？		
它有沒有 跟專案既有寫法不一致 ？		
你要花多久才能確認它是對的？		

實務 進階 同一個需求，把 `git reset` 之後**再跑一次**，比較兩次的結果差多少——這就是「同樣輸入不保證同樣輸出」的實測。

第 3、4 列才是重點

它漏掉的，跟它自作主張的，
都來自同一個原因：

你覺得理所當然的事，它不知道。

- 要不要加錯誤處理？命名跟哪個檔一致？要不要寫測試？
- 沒講的部分，它用**自己的品味**決定
- 而那個品味來自全世界的程式碼，不是這個專案

重點 下一節要做的事很簡單：把「理所當然」寫下來，讓它每次都看得到。

收斂：context 的四個來源

Lab 1 之後，我們把「該給什麼」整理成四類。

來源	內容	這門課在哪裡處理
你當下打的字	這次的具體要求、格式、邊界	節 1
常駐的設定檔	專案規範、慣例、禁止事項	節 2
工具抓回來的	讀檔、查資料庫、呼叫 API	節 2
外部知識庫	文件、wiki、內部規定	節 3

重點 你今天只用了第一種。接下來三節，我們把另外三種一個一個接上。

你有 agent 了，但它還不知道你的規矩

節 1 小結

你今天學到

- agent 是派工，不是問答
- 它的錯誤是有規律的
- 你覺得理所當然的，它不知道
- prompt / context / harness / loop

你今天做到

- 跑起了自己的 coding agent
- 用它做出一個真的功能
- 填出了那張觀察記錄表

下一節：Harness Engineering

開始動模型**外面**的東西：

- 每次開機就知道專案規矩
- 給它工具去碰外部系統
- 做完自動被檢查

實務 記錄表第 3、4 列的內容，下一節會變成 agent **每次啟動都自動載入**的規矩。